SESSIONAL SERIES · DEPARTMENT OF CSE

Defining & Changing the Structure

CSE 2206 · Lecture 02 — Data Definition Language in PostgreSQL

Saad Ahmed
Lecturer, Dept. of CSE, BAUST

Md. Rakib Trofder
Lecturer, Dept. of CSE, BAUST

CREATE TABLE · ALTER TABLE · DROP TABLE · SCHEMAS · DATABASES

WHAT WE COVER TODAY

Today's Roadmap

Six steps, all performed on the Library table you built in Lecture 01.

1 CREATE TABLE

Build a table with types, defaults and constraints from the start.

2 Add & Remove Columns

ALTER TABLE ADD COLUMN and DROP COLUMN on a live table.

3 Rename & Retype

Change a column name, and change its data type safely.

4 Constraints

Make a column the primary key, or make it NOT NULL.

5 DROP TABLE

Remove the structure — and how that differs from DELETE.

6 Schemas & Databases

Create, rename and drop the two containers above the table.

TWO KINDS OF SQL

DDL Changes the Shape, DML Changes the Rows

Lecture 01 was all DML. Today every statement rewrites the definition of the table itself.

DDL — Data Definition

Today

- CREATE TABLE — bring a new table into existence.
- ALTER TABLE — add, drop, rename or retype a column.
- DROP TABLE — remove the table and everything in it.
- CREATE / ALTER / DROP for schemas and databases.

DML — Data Manipulation

Lecture 01

- SELECT … WHERE … ORDER BY … LIMIT — read rows.
- INSERT — add rows to an existing table.
- UPDATE — change values inside existing rows.
- DELETE — remove rows, leaving the table behind.

Useful in PostgreSQL: DDL is transactional. BEGIN; ALTER TABLE …; ROLLBACK; really does put the old structure back — practise every risky change that way. (MySQL and Oracle do not allow this.)

BUILDING THE TABLE

CREATE TABLE

Name, columns, types — and any rule that must be true from the very first row.

THE LIBRARY TABLE, DEFINED PROPERLY

```
CREATE TABLE IF NOT EXISTS Library (
    BookID          INT PRIMARY KEY,
    BookTitle       VARCHAR(120) NOT NULL,
    Author          VARCHAR(100),
    Category        VARCHAR(50),
    Publisher       VARCHAR(100),
    AvailableCopies INT DEFAULT 0
);
```

Table and column names are folded to lower case unless you wrap them in double quotes.

IF NOT EXISTS

Runs without error when the table is already there — a notice instead of a failure.

PRIMARY KEY

Unique and never NULL. Only one per table; PostgreSQL builds an index for it.

NOT NULL

The column must always carry a value — an INSERT that omits it is rejected.

DEFAULT 0

The value used when an INSERT does not mention the column.

BEFORE YOU CHANGE ANYTHING

Read the Current Structure

Every ALTER in this lecture starts by looking at what is actually there.

`\d library`

Column	Type	Nullable	Default
bookid	integer	not null	
booktitle	character varying(120)	not null	
author	character varying(100)		
category	character varying(50)		
publisher	character varying(100)		
availablecopies	integer		0

```
Indexes:  "library_pkey" PRIMARY KEY, btree (bookid)
```

QUICK LOOKUPS

`\dt`	which tables exist here
`\d library`	columns, types, constraints
`\dn`	which schemas exist
`\l`	which databases exist

THE PORTABLE VERSION

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'library';
```

ONE STATEMENT, MANY ACTIONS

ALTER TABLE

Everything on the next six slides is the same statement with a different action at the end.

```
ALTER TABLE Library  <action>;
```

ADD COLUMN

```
Edition INT
```

DROP COLUMN

```
Edition
```

RENAME COLUMN

```
Author TO AuthorName
```

ALTER COLUMN … TYPE

```
Category TYPE VARCHAR(80)
```

ADD CONSTRAINT … PRIMARY KEY

```
(BookID)
```

ALTER COLUMN … SET NOT NULL

```
BookTitle
```

The table keeps its rows through all of these — only the definition changes.

GROWING THE TABLE

ADD COLUMN

A new column is appended at the end and filled with NULL — or with a DEFAULT if you give one.

ONE COLUMN

```
ALTER TABLE Library ADD COLUMN Edition INT;
```

WITH A DEFAULT AND A RULE

```
ALTER TABLE Library
ADD COLUMN Price NUMERIC(8,2) DEFAULT 0 NOT NULL;
```

TWO COLUMNS IN ONE STATEMENT

```
ALTER TABLE Library
  ADD COLUMN ISBN VARCHAR(20),
  ADD COLUMN ShelfNo VARCHAR(10);
```

AFTER THE FIRST STATEMENT

BookTitle	Edition
Database System Concepts	NULL
Computer Networks	NULL
Operating System Concepts	NULL
Introduction to Algorithms	NULL
Clean Code	NULL

ADD COLUMN … NOT NULL with no DEFAULT fails on a table that already has rows — there is nothing to put in the existing ones. Give a DEFAULT, or fill the column with UPDATE first.

SHRINKING THE TABLE

DROP COLUMN

The column and every value inside it are removed from all five rows at once.

REMOVE ONE COLUMN

```
ALTER TABLE Library DROP COLUMN Edition;
```

DO NOT FAIL IF IT WAS NEVER THERE

```
ALTER TABLE Library DROP COLUMN IF EXISTS ShelfNo;
```

TWO AT ONCE

```
ALTER TABLE Library
  DROP COLUMN ISBN,
  DROP COLUMN Price;
```

Before dropping, SELECT the column once and keep the output — it is the cheapest backup you will ever make.

There is no undo

DROP COLUMN destroys the data in that column for every row. Once the transaction is committed, only a backup brings it back.

WHAT CAN BLOCK IT

- A view or foreign key that uses the column — PostgreSQL refuses.
- Adding CASCADE drops those dependents too. Check what they are first.
- Dropping the primary key column also removes the primary key.

CHANGING NAMES

RENAME COLUMN & RENAME TABLE

Renaming touches only the name. Types, data and constraints all stay exactly as they were.

RENAME A COLUMN

```
ALTER TABLE Library
RENAME COLUMN Author TO AuthorName;
```

RENAME THE WHOLE TABLE

```
ALTER TABLE Library
RENAME TO BookCatalogue;
```

PUT IT BACK

```
ALTER TABLE BookCatalogue
RENAME TO Library;
```

Data is untouched

All five rows keep their values — only the header changes.

One name per statement

RENAME cannot be combined with ADD or DROP in the same ALTER.

Your old queries break

Anything referring to Author now fails. Views and application code must be updated too.

CHANGING THE TYPE

ALTER COLUMN … TYPE

PostgreSQL rewrites every existing value into the new type — so it must be able to convert them all.

WIDENING — ALWAYS SAFE

```
ALTER TABLE Library
ALTER COLUMN Category TYPE VARCHAR(80);
```

NUMBER TO TEXT — AUTOMATIC

```
ALTER TABLE Library
ALTER COLUMN AvailableCopies TYPE VARCHAR(10);
```

TEXT BACK TO NUMBER — NEEDS USING

```
ALTER TABLE Library
ALTER COLUMN AvailableCopies TYPE INT
USING AvailableCopies::INT;
```

WHEN IT WORKS BY ITSELF

- VARCHAR(50) → VARCHAR(80): more room, no risk.
- INT → NUMERIC or INT → VARCHAR: every value converts.
- Same family, larger size: no USING needed.

When it fails

Shrinking VARCHAR(120) to VARCHAR(20) fails if one title is longer. Text to INT fails if one value is not a number — and the whole statement is rolled back, not just that row.

ADDING A CONSTRAINT

Making a Column the Primary Key

A primary key is a named constraint you can add to, and remove from, an existing table.

ADD IT TO AN EXISTING COLUMN

```
ALTER TABLE Library
ADD CONSTRAINT pk_library PRIMARY KEY (BookID);
```

REMOVE IT AGAIN

```
ALTER TABLE Library DROP CONSTRAINT pk_library;
```

A KEY OVER TWO COLUMNS

```
ALTER TABLE Library
ADD CONSTRAINT pk_lib PRIMARY KEY (BookID, Publisher);
```

THREE CONDITIONS

1. **The column must be NOT NULL**
   PostgreSQL sets this for you when it can — otherwise set it first.

2. **Every value must be unique**
   One duplicate BookID and the statement is refused.

3. **Only one per table**
   Drop the existing constraint before adding a different one.

Name your constraints: pk_library is far easier to drop than a generated name.

ADDING A CONSTRAINT

SET NOT NULL & DROP NOT NULL

NOT NULL is a property of the column, so it is set with ALTER COLUMN — not with ADD CONSTRAINT.

REQUIRE A VALUE

```
ALTER TABLE Library
ALTER COLUMN BookTitle SET NOT NULL;
```

ALLOW NULL AGAIN

```
ALTER TABLE Library
ALTER COLUMN BookTitle DROP NOT NULL;
```

A DEFAULT VALUE FOR THE COLUMN

```
ALTER TABLE Library
ALTER COLUMN AvailableCopies SET DEFAULT 0;
```

Existing NULLs block it

```
UPDATE Library SET Publisher = 'Unknown'
WHERE Publisher IS NULL;
```

Fill the gaps first, then run SET NOT NULL — otherwise the statement is refused.

NOT NULL vs DEFAULT

- NOT NULL rejects the INSERT when a value is missing.
- DEFAULT quietly supplies a value instead.
- Together: the column always has a sensible value, and never a NULL.

REMOVING THE TABLE

DROP TABLE

DELETE empties the table. DROP removes the table itself — columns, constraints, indexes and all.

REMOVE THE TABLE

```
DROP TABLE Library;
```

SAFE IN A SCRIPT

```
DROP TABLE IF EXISTS Library;
```

TAKE DEPENDENT VIEWS WITH IT

```
DROP TABLE Library CASCADE;
```

THREE WAYS TO EMPTY OR REMOVE

- **`DELETE FROM Library;`**
  Rows go, table stays. Can be filtered with WHERE.
- **`TRUNCATE Library;`**
  All rows go at once, faster, no WHERE allowed.
- **`DROP TABLE Library;`**
  Rows and structure go. SELECT now says table does not exist.

Practise it safely: BEGIN; DROP TABLE Library; … then ROLLBACK; and the table is still there. Get into this habit before you ever touch a real database.

THE CONTAINER ABOVE THE TABLE

Create, Rename & Drop a Schema

A schema is a folder for tables inside one database. Everything so far lived in public.

```
CREATE SCHEMA campus;
```

Make a new namespace

```
ALTER SCHEMA campus RENAME TO academics;
```

Rename it

```
DROP SCHEMA academics;
```

Only works when it is empty

```
DROP SCHEMA academics CASCADE;
```

Removes every table inside it too

```
ALTER TABLE Library SET SCHEMA academics;    -- move a table
```

Why schemas exist

- Two tables may share a name in different schemas.
- Permissions can be granted per schema.
- public is created with every new database.

WHICH SCHEMA AM I IN?

```
SHOW search_path;

SET search_path TO academics, public;

SELECT * FROM academics.Library;
```

THE CONTAINER ABOVE THE SCHEMA

Create, Rename & Drop a Database

The largest container. These statements run while you are connected to some other database.

```
CREATE DATABASE libraryDB;
```

A fresh database with only the public schema

```
ALTER DATABASE libraryDB RENAME TO campusDB;
```

Rename it — nobody may be connected

```
DROP DATABASE IF EXISTS campusDB;
```

Removes every schema, table and row inside

TYPICAL PSQL SESSION

```
\c postgres          -- leave the database first
DROP DATABASE campusDB;
```

Disconnect first

You cannot rename or drop the database you are currently connected to.

Not inside a transaction

CREATE and DROP DATABASE cannot run inside BEGIN … COMMIT — there is no rollback here.

Everything goes

Dropping a database removes all its schemas, tables, rows and indexes.

EVERYTHING ON ONE PAGE

DDL Cheat Sheet

The whole lecture, in the order you will usually need it.

Goal	Statement	Watch out for
Create a table	`CREATE TABLE IF NOT EXISTS Library ( … );`	Constraints apply from row one
Add a column	`ALTER TABLE Library ADD COLUMN Edition INT;`	New column is filled with NULL
Add with a default	`ALTER TABLE Library ADD COLUMN Price NUMERIC(8,2) DEFAULT 0;`	Needed if you also want NOT NULL
Remove a column	`ALTER TABLE Library DROP COLUMN Edition;`	The data goes with it
Rename a column	`ALTER TABLE Library RENAME COLUMN Author TO AuthorName;`	Old queries stop working
Rename the table	`ALTER TABLE Library RENAME TO BookCatalogue;`	Same warning
Change the type	`ALTER TABLE Library ALTER COLUMN Category TYPE VARCHAR(80);`	Add USING for risky casts
Add a primary key	`ALTER TABLE Library ADD CONSTRAINT pk_library PRIMARY KEY (BookID);`	Must be unique and NOT NULL
Require a value	`ALTER TABLE Library ALTER COLUMN BookTitle SET NOT NULL;`	Existing NULLs block it
Delete the table	`DROP TABLE IF EXISTS Library;`	Different from DELETE
Schema	`CREATE / ALTER … RENAME TO / DROP SCHEMA campus;`	DROP needs it empty, or CASCADE
Database	`CREATE / ALTER … RENAME TO / DROP DATABASE libraryDB;`	Disconnect from it first

DO IT YOURSELF

Lab Tasks

Work on your own copy of Library. Run \d library after every step and note what changed.

1. Create a schema called campus and a table Library inside it.
2. Add a column Edition INT and a column Price NUMERIC(8,2) DEFAULT 0.
3. Rename Author to AuthorName, then rename the table to BookCatalogue and back.
4. Change Category to VARCHAR(80), then try shrinking BookTitle to VARCHAR(10) and explain the error.
5. Drop the primary key, then add it back as pk_library.
6. Make BookTitle NOT NULL, then prove it by attempting an INSERT without a title.
7. Drop the Edition column, then drop the whole table inside a transaction and ROLLBACK.
8. Rename schema campus to academics, then drop it with CASCADE.

Submit one .sql file with all eight tasks, each preceded by a comment naming the task.

WRAP-UP

What You Can Do Now

Create

CREATE TABLE with types, DEFAULT and constraints

Reshape

ADD, DROP, RENAME and retype a column

Constrain

PRIMARY KEY, NOT NULL and DEFAULT after the fact

Contain

CREATE / ALTER / DROP for schemas and databases

Next session

FOREIGN KEY, UNIQUE and CHECK constraints · a second table · joining Library with Borrower



Saad Ahmed · Md. Rakib Trofder — Department of CSE, BAUST